

UNIDAD 5

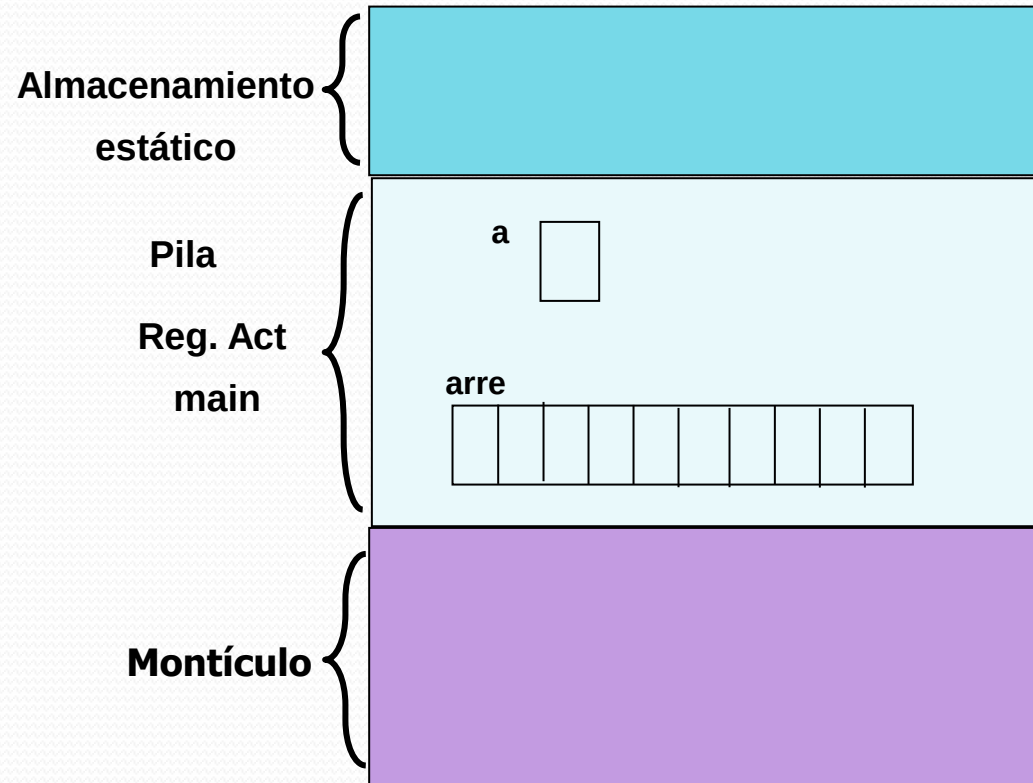


Programación Procedural Estructuras Dinámicas

Estructuras de Datos Dinámicas

Variables con Almacenamiento fijo o estático

```
void main ()  
{  
    float a;  
    int arre[10];  
    .....  
}
```



Estructuras de Datos Dinámicas

Pueden *crearse en tiempo de ejecución* y su espacio de almacenamiento pueda *liberarse cuando no se las necesite* y modificar su tamaño durante la ejecución del programa.

Objetos de datos dinámicos
(*variables o estructuras dinámicas*)



Punteros

Almacenamiento Estático y Almacenamiento Dinámico

Estáticas o de Almacenamiento Fijo:

- **El sistema asigna el área que ocupará, a partir de su declaración** (identificador y tipo)
- El almacenamiento permanece fijo durante la ejecución de la función que la declara
- No se pueden utilizar esas posiciones de memoria para otros fines durante la ejecución

Dinámicas:

- **Se crean en tiempo de ejecución**, en cualquier punto, no sólo al entrar a un subprograma, **a petición**
- Se puede liberar su espacio de almacenamiento cuando no se las necesite
- En el caso de estructuras, se puede modificar su tamaño durante la ejecución del programa.

Variables Dinámicas: Montículo o montón

Se crean y se libera por petición, **durante la ejecución del programa** y no en forma automática como las estáticas o de almacenamiento estático.

**Almacenamiento
estático**

**Pila o
Stack**

**Montículo o
heap**

Segmento de códigos de
funciones y datos

Almacenamiento de
registros de activación

**Almacenamiento
dinámico**

Organización de la memoria durante la ejecución de un programa

Montículo o heap

Operaciones

pueden ser



Alojamiento: Se demanda un bloque para almacenar un objeto de cierto tamaño. Resultado de la función es un puntero a un trozo contiguo de memoria dentro del montículo

Desalojo: Se indica que ya no es necesario conservar un objeto, la memoria que ocupa queda libre para ser reutilizada

Explícitas: el **programador** invoca operaciones (*lenguajes imperativos*), incluye en código fuente una instrucción que **demanda cierta cantidad de memoria para** la ubicación de un dato.

malloc en C

Implícitas: el **sistema** realiza las operaciones (*lenguajes lógicos y funcionales*), la memoria requerida es calculada por el compilador en función del tipo correspondiente al objeto que se desea alojar.

Características del lenguaje para definir variables dinámicas

Poseer un **tipo elemental de datos** apuntador, llamado también tipo de referencia o de acceso. (puntero)

Una **operación de creación de objetos** de tamaño fijo, que devuelve el **valor L** del bloque de almacenamiento creado y ese valor L se convierte en el valor **r** de una variable del tipo apuntador.

C: malloc

Una **operación de desreferenciar** para objetos del tipo apuntador, que permite acceder al valor del objeto al cual apunta.

Características e implementación de punteros

Apuntadores pueden

- referenciar objetos de un único tipo de datos
C: malloc (lenguajes utilizan declaración de tipo y verificación estática)
- referenciar a objetos de cualquier tipo de datos
SmallTalk (los objetos utilizan descriptores y se realiza verificación dinámica de tipos)

Implementación

Direcciones absolutas: El apuntador contiene la **dirección de memoria real** del bloque de almacenamiento del objeto.

Direcciones relativas: El apuntador contiene **el desplazamiento del objeto respecto a la dirección base** de un bloque de almacenamiento más grande.

Manejo del Montículo en lenguaje C

Funciones malloc y free

Función malloc (Memory ALLOCation: asignación de memoria):

Solicita a sistema operativo un bloque de memoria en el montículo, del tamaño especificado en el argumento.

Resultado: dirección de comienzo del bloque asignado o la constante NULL, si no hay espacio disponible. El tipo del bloque asignado es Void. (se le debe hacer un cast al tipo usado)

Formato: malloc (tamaño del bloque)

Función free Libera bloques asignados previamente por malloc. De esta manera existe más espacio de memoria disponible para posteriores asignaciones.

Formato: free(variable puntero);

Biblioteca en Dev C++ **#include <malloc.h>**

Sistema Operativo y Gestión de Asignación de Memoria

- El Sistema Operativo maneja una tabla de direcciones de memoria que indican el espacio de memoria ocupado (para que un programa no modifique direcciones de memoria que no le corresponden, corrompiendo la memoria de otros procesos / programas / información).
- **malloc** indica que se va a cargar en memoria una estructura y por ello el sistema operativo registra en su tabla de direcciones de memoria que un bloque ha sido ocupado, y envía a malloc la dirección de inicio de ese bloque, de tipo void por lo cual se debe hacer un cast antes de ser asignada a una variable puntero.

var_puntero= (tipo_de_dato_de_var_puntero*) malloc (tamaño del bloque)


Cast (conversión explícita)

- **free** indica que el bloque de memoria asignado a través de malloc se ha dejado de usar, con esta información el Sistema Operativo registra en su tabla de direcciones de memoria que dicho bloque ha sido desocupado. La variable puntero conserva el valor que tenía antes de free, es decir no cambia su contenido, por este motivo, para evitar errores de acceso, resulta ser una buena práctica inicializar la variable después de su liberación con la constante **NULL**.

Funciones malloc y free en C

EJEMPLO

```
#include <malloc.h>
```

```
struct alumno
```

```
{
```

```
    char nom[20];
```

```
    float prom;
```

```
};
```

```
int main()
```

```
{
```

```
    int *n, num;
```

```
    alumno *p;
```

```
    printf("\nIngrese un número entero");
```

```
    scanf("%d",&num);
```

```
    n = (int *) malloc (sizeof (int));
```

```
    *n = num;
```

```
    p=(alumno *) malloc (sizeof (alumno));
```

```
    fflush(stdin);
```

```
    printf("\nIngrese nombre");
```

```
    gets(p-> nom);
```

```
    printf("\nIngrese promedio");
```

```
    scanf("%f", &p-> prom);
```

```
    printf("\n Número entero es %d", *n);
```

```
    printf("\n el alumno ingresado es %s y tiene  
           promedio %f", p-> nom, p-> prom);
```

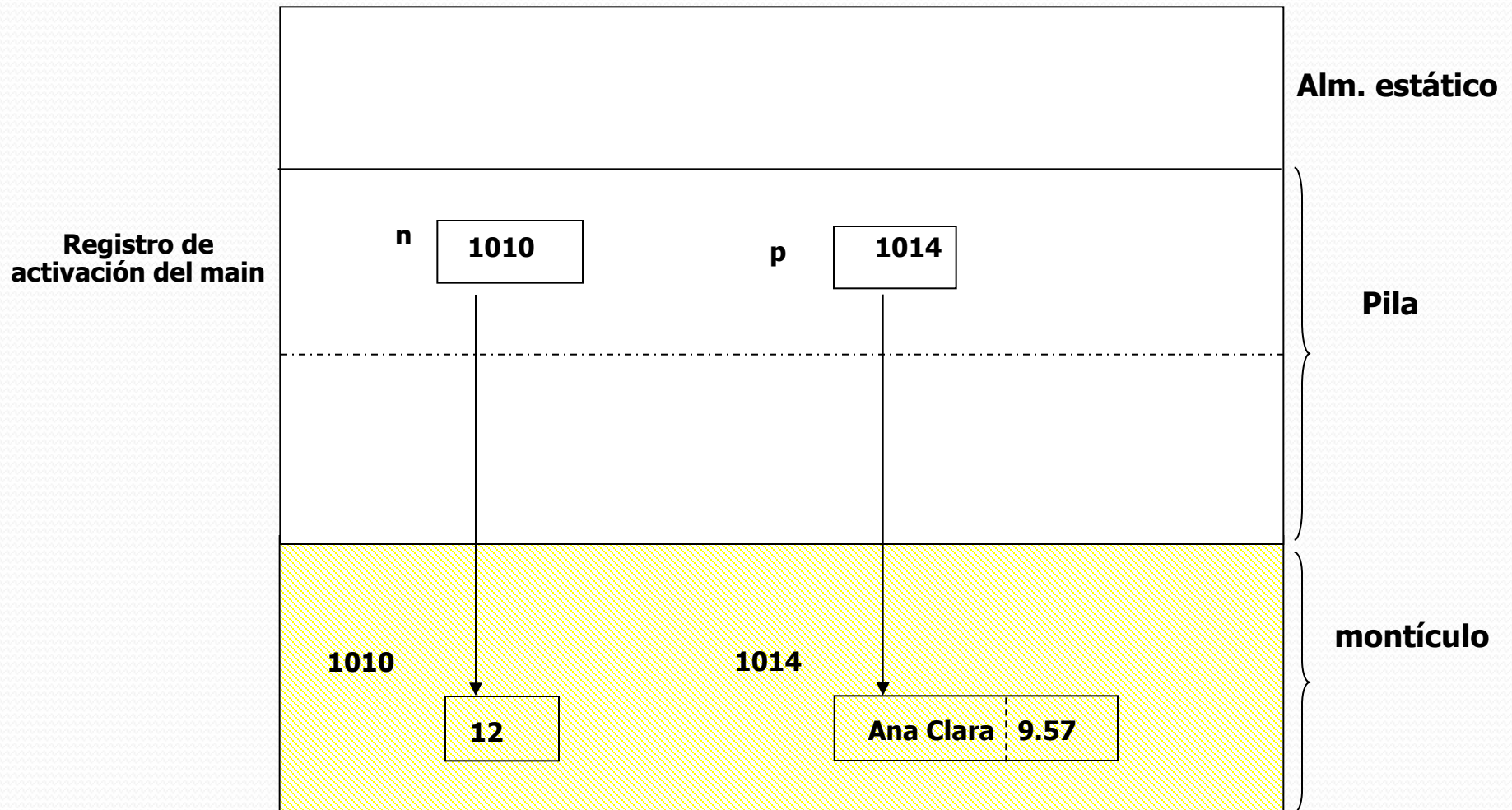
```
    free(n);
```

```
    free(p)
```

```
    getch();
```

```
}
```

Funciones malloc y free



Funciones malloc y free en C

Algunos Inconvenientes que pueden surgir:

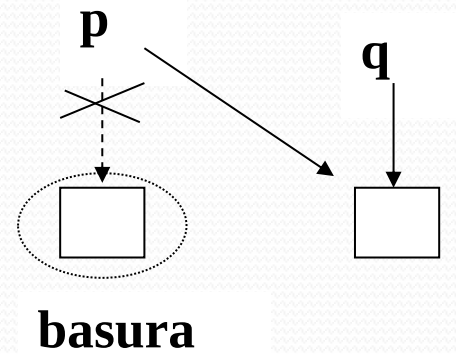
Presencia de elemento basura

Presencia de referencias desactivadas

Presencia de elemento basura: cuando habría espacio disponible para nuevo uso, pero dicho espacio no está en la lista de espacios libres, por ello se ha vuelto inaccesible.

Ejemplo :

```
int *p, *q;  
q=(int *) malloc(sizeof (int));  
p=(int *)malloc(sizeof (int));  
.....  
p=q;
```





☐ Nuevo ☐ Insertar ☒ Activar/Desactivar ☐ Ir a

p [*] Sin Nombre1 BASURA.cpp

```
# include <stdio.h>
#include <malloc.h>
int main(void)
{
    int *p,*q;
    q=(int *)malloc(sizeof (int));
    p=(int *)malloc(sizeof (int));
    *p= 5;
    *q= 10;
    printf(" \n %d %d", *p,*q);
    p=q;
    printf(" \n %d %d", *p,*q);
    getchar();
}
```

D:\IVO\PROCEDURAL 2014\UNIDAD 5\Ejemplos de teoría\BASURA.exe

```
5 10
10 10_
```

Registro de Compilación ☒ Depuración ☐ Resultados

16 Líneas en Archivo



Funciones malloc y free en C

Referencias desactivadas

Esto sucede cuando al finalizar el tiempo de vida de un objeto, el almacenamiento se recupera para ser reasignado, pero no se destruyen todas las rutas de acceso.

Ejemplo:

```
void modifica (int *&p)
{
    int *x;
    x=(int*)malloc(sizeof(int));
    *x=25;
    p=x;
    free(x);
}
```

El uso de p después de ejecutar la función *modifica* puede acarrear problemas

```
C:\Users\Mario\Documents\A - Facultad\0 Procedural 2019\Si...
valor del puntero n a21420 ***** variable dinamica (lo apuntado por n) 3
valor de los punteros p: a21420 n: a21420
valor de puntero n a21420 lo apuntado por n (despues de free) 10617168
valor de puntero p a21420 lo apuntado por p (despues del free de n) 10617168
valor del puntero despues de asignar NULL 0
-----
Process exited after 0.05112 seconds with return value 0
Presione una tecla para continuar . . .
```

Basura –
bloque de
memoria
usado por
el SO

EJEMPLO

```
#include <malloc.h>
```

```
int main()
```

```
{
```

```
    int *n,*p;
```

```
    n= (int*) malloc( sizeof(int));
```

```
    *n=3;
```

```
    printf("\n valor del puntero %x ***** variable dinamica %5d",n,*n);
```

```
    p=n;
```

```
    printf("\n valor de los punteros %5x %5x",p, n);
```

```
    free(n);
```

```
    printf("\n valor de puntero %5x variable después de liberar puntero%5d ",n, *n);
```

```
    printf("\n valor de puntero %5x variable después de liberar el puntero% 5d ",p, *p);
```

```
    p=n=NULL;
```

```
    printf("\n valor del puntero después de asignar NULL %5x ",p);
```

```
}
```